



LangChain

Tools Handbook



1. tools overview

19 de septiembre de 2026



Contenido

0. Tools & Tool Calling Overview

1. Simple Definition

add is now a Tool the model can be given and can choose to call.

2. Why Does Tool Calling Exist?

Before tool calling (the model guesses)

→ **"Approximately 239 million" — often WRONG; the model isn't a calculator.**

→ **"I can't access real-time data." — it simply can't.**

After tool calling (the model delegates)

Give it a calculator tool and a weather tool.

For math → it calls `multiply(24387, 9823)` → gets the EXACT answer.

For weather → it calls `get_weather('Paris')` → gets LIVE data.

3. Real-Life Analogy

4. Where Tools Fit in LangChain Architecture

5. Internal Working — the full loop

6. The Core Vocabulary

Tool

`bind_tools()`

`tool_call`

`ToolMessage`

7. A crucial safety point

0. Tools & Tool Calling Overview

Before the individual pieces, let's understand **what a tool is**, **why tool calling exists**, and the **full loop** from user question to final answer. Learn this once and every later file clicks.

1. Simple Definition

Kid version: Imagine a super-smart friend who's been locked in a room since 2023. They know a lot, but they can't look out the window, use a calculator, or check your fridge. So you give them a **walkie-talkie** connected to helpers outside. When they need something they can't do, they radio: "Please check the weather!" A helper does it and radios back the answer.

- The **helpers** (weather-checker, calculator, fridge-looker) = **tools**.
- The **walkie-talkie routine** of asking and getting answers = **tool calling**.

Professional definition: A *tool* is a function, paired with a name and a description, that an LLM can choose to invoke. *Tool calling* is the protocol where the model outputs a structured request to run a tool with specific arguments; your application executes it and returns the result to the model.

```
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

# `add` is now a Tool the model can be given and can choose to call.
```

2. Why Does Tool Calling Exist?

The problem: An LLM has three big limitations:

1. **Frozen knowledge** — it doesn't know today's news, prices, or your private data.
2. **No actions** — it can't send an email, run code, or write to a database.
3. **Unreliable at exact tasks** — math, counting, precise lookups (it *predicts* text, it doesn't *compute*).

Tools fix all three by letting the model **delegate** to real code.

Before tool calling (the model guesses)

```
model.invoke("What is 24387 * 9823?")
# → "Approximately 239 million" – often WRONG; the model isn't a calculator.

model.invoke("What's the weather in Paris right now?")
# → "I can't access real-time data." – it simply can't.
```

After tool calling (the model delegates)

```
# Give it a calculator tool and a weather tool.
# For math → it calls multiply(24387, 9823) → gets the EXACT answer.
# For weather → it calls get_weather('Paris') → gets LIVE data.
```

The model becomes a **router/orchestrator**: it decides *which* capability is needed and *with what arguments*, while real code does the precise work.

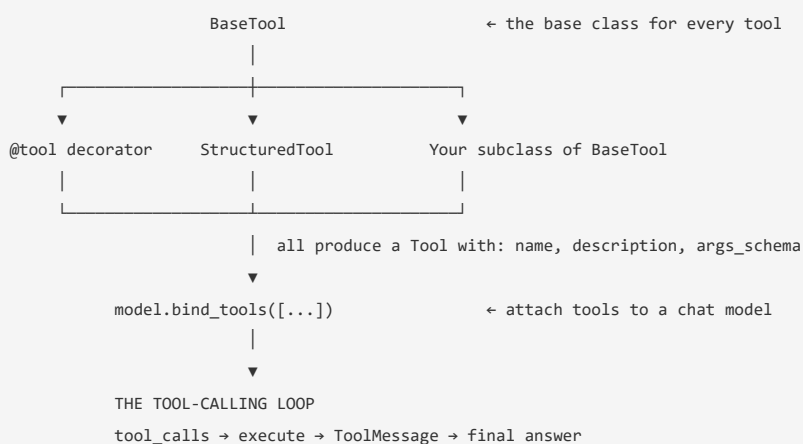
Where you'll use it: agents, RAG that can search, database Q&A, calculators, API integrations, booking/ordering assistants, code execution — anything where the LLM must *do* rather than only say.

3. Real-Life Analogy

A **manager delegating to specialists** 🧑‍💼. A good manager doesn't personally do every task — they know *who* to ask. "Accounting, run these numbers." "IT, restart the server." The manager (LLM) reads the situation and delegates to the right specialist (tool), then combines the results into a decision.

Other analogies: a **doctor ordering lab tests** (they interpret results but don't run the machines), a **chef calling out to line cooks**, a **detective sending evidence to the forensics lab**.

4. Where Tools Fit in LangChain Architecture

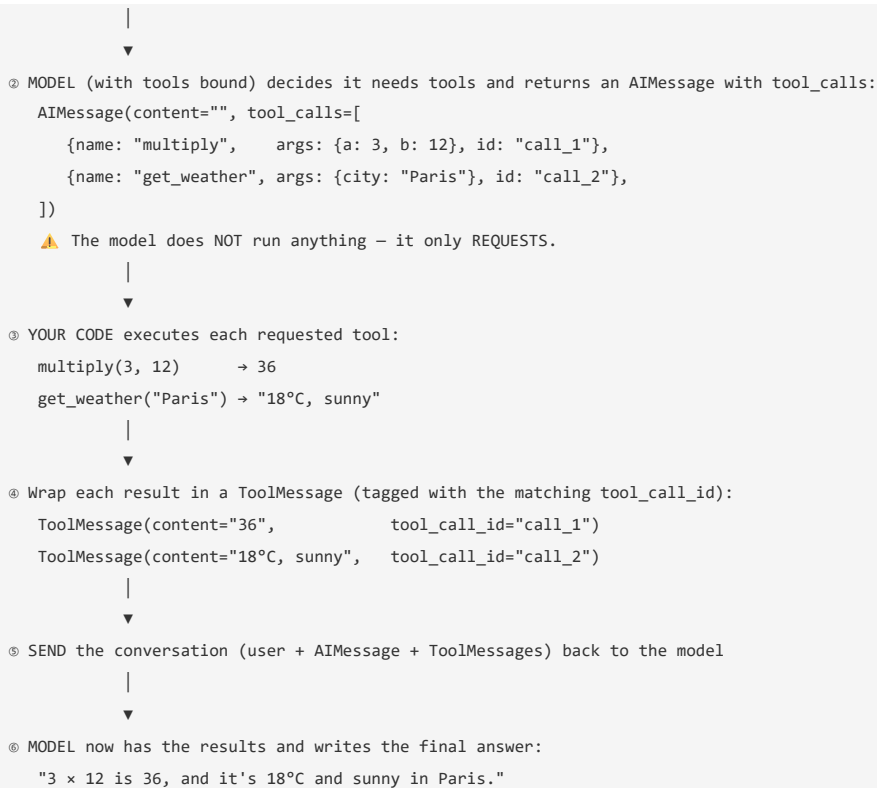


Three ways to make a tool, one way to describe it to the model, one way to attach it, and one loop to run it.

5. Internal Working — the full loop

This is the single most important diagram in the notebook:

```
① USER asks something
   "What's 3 times 12, and the weather in Paris?"
```



Key insight: it's a **two-round-trip** conversation. Round 1 the model asks for tools; round 2 it *answers* using the results. An **agent** just automates this loop, possibly for many rounds.

6. The Core Vocabulary

Tool

Definition: A callable + `name` + `description` + `args_schema` that the model can invoke.

Why it exists: It's the unit of capability you give the model.

Real-life use case: One specialist the manager can delegate to.

```
@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city."""
    return f"18°C and sunny in {city}"
```

bind_tools()

Definition: Attaches a list of tools to a chat model so it *can* call them.

Why it exists: The model must be told which tools exist (and their schemas) before it can request them.

```
model_with_tools = model.bind_tools([get_weather, add])
```

tool_call

Definition: The model's structured **request** to run a tool: `{name, args, id}`. Found on `AIMessage.tool_calls`.

Why it exists: It's how the model communicates "run this, with these arguments."

Real-life use case: The written work-order the manager hands a specialist.

```
ai_msg = model_with_tools.invoke("weather in Paris?")
ai_msg.tool_calls  # [{'name': 'get_weather', 'args': {'city': 'Paris'}, 'id': 'call_abc'}]
```

ToolMessage

Definition: The message carrying a tool's **result** back to the model, tagged with `tool_call_id`.

Why it exists: So the model can match each result to the request it made.

Real-life use case: The specialist's report sent back to the manager.

```
from langchain_core.messages import ToolMessage
ToolMessage(content="18°C and sunny", tool_call_id="call_abc")
```

7. A crucial safety point

The model never executes your code. It only *emits a request* (`tool_calls`). **Your program** decides whether and how to run it. This means:

- You can validate/authorize before running (e.g. don't let it delete files).
- You can sandbox dangerous tools (code execution, shell).
- You stay in control — the LLM is an advisor, not an admin.

2. builtin tools and toolkit

19 de septiembre de 2026



Contenido

Built-in Tools & Toolkits

1. Simple Definition
2. Why Do They Exist?
 - Before (roll your own web search)
 - After (use a built-in)
3. Real-Life Analogy
4. Where They Fit in LangChain Architecture
5. Commonly used built-in tools
 - Web search (Tavily / DuckDuckGo / SerpAPI)
 - Python REPL
 - Wikipedia

Built-in Tools & Toolkits

You don't have to build every tool yourself. LangChain ships **ready-made tools** (web search, Python execution, Wikipedia, etc.) and **toolkits** (bundles of related tools for a system like a database or Gmail). Reuse before you reinvent.

1. Simple Definition

Kid version: You don't need to build every tool from scratch. But the store already sells lots of gadgets — a web-searcher, a calculator, a Wikipedia-reader. A **toolkit** is a whole boxed set of matching gadgets for one job (like a full "database repair kit"). Grab those instead of building from scratch.

Professional definition:

- A **built-in tool** is a pre-implemented `BaseTool` provided by LangChain or its integration packages (e.g., search, REPL).
- A **toolkit** is a class that groups related tools for a specific system, exposing them via `.get_tools()`.

```
from langchain_community.tools import DuckDuckGoSearchRun

search = DuckDuckGoSearchRun()
search.invoke("latest langchain release")  # returns search results as a string
```

2. Why Do They Exist?

The problem: Common capabilities (web search, Wikipedia, running Python, querying SQL) are needed by *everyone*. Rewriting them — with correct schemas, error handling, and API plumbing — wastes time and invites bugs.

Before (roll your own web search)

```
@tool
def web_search(query: str) -> str:
    """Search the web."""
    # sign up for an API, handle auth, pagination, rate limits, parsing... 🤖
    ...
```

After (use a built-in)

```
from langchain_community.tools.tavily_search import TavilySearchResults
search = TavilySearchResults(max_results=3)  # done - battle-tested
```

Built-ins are already `BaseTool`s, so they drop straight into `bind_tools([...])` and the loop.

3. Real-Life Analogy

IKEA vs. building furniture from raw timber 🪚. A **built-in tool** is a ready-made chair. A **toolkit** is a full flat-pack set (table + 4 matching chairs) designed to work together for one room (one system, like "your SQL database"). You *can* carve your own, but usually you shouldn't.

4. Where They Fit in LangChain Architecture

```

langchain_community / integration packages
|
├─ built-in tools    (DuckDuckGoSearchRun, WikipediaQueryRun, PythonREPLTool, ...)
|   └─ each IS a BaseTool → drop into bind_tools([...])
|
└─ toolkits          (SQLDatabaseToolkit, GmailToolkit, FileManagementToolkit, ...)
    └─ .get_tools() → returns a LIST of related BaseTools
  
```

Everything ends up as `BaseTool`s feeding into `bind_tools` and the loop.

🍪 Many built-ins live in `Langchain_community` or provider packages (e.g. `Langchain_tavily`). Install the relevant package; APIs move around across versions, so check imports for your version.

5. Commonly used built-in tools

Web search (Tavily / DuckDuckGo / SerpAPI)

Definition: Query a web-search API and return results as text.

Why it exists: Give the model live, post-training-cutoff information.

When developers use it: Research assistants, "current events" Q&A, RAG-with-web.

```

In [1]:
from langchain_community.tools import DuckDuckGoSearchRun
search_tool = DuckDuckGoSearchRun()
result = search_tool.invoke("Tell me about CJP protest")
print(result)
  
```

```

Out [1]:
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_25488\575319334.py:1: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-community/issues/674 for details and migration guidance toward standalone integration packages.
  from langchain_community.tools import DuckDuckGoSearchRun
2 weeks ago - The agitation drew national attention ... at the protest site on 28 June. The CJP had been created in May 2026 by digital communications strategist Abhijeet Dipke as a political satire, alluding to a statement by Surya Kant, the Chief Justice of India, who compared unemployed youth and social media activists ... 4 days ago - Dipke said that the CJP is not ... of protests, including from left-wing student organisations like AISF and AIYF, the chief of right-wing Hindutva organization, Rashtriya Swayamsevak Sangh (RSS), and political organizations, Aam Aadmi Party, Shiv Sena and Communist Party of India (Marxist), among others. The CJP created a mock political platform that rapidly gained traction across social media... The CJP is mostly protesting over issues they have with India's education system. The National Eligibility cum Entrance Test (NEET) is India's main entrance exam for admission to medical colleges across the country. Published: July 23, 2026 The CJP protest began with exam paper leaks. It became India's biggest student movement in years - why? Published: July 25, 2026 August 10, 2026 - So me, Ashutosh, and Vijeta were first announced when Abhijeet was still in the US. And after that, we had other people coming in publicly. CJP co-convenors Saurav Das and Ashutosh Ranka at the protest after Education Minister Dharmendra Pradhan's resignation, in New Delhi on June 25. | Photo Credit: Vipin Kumar/ANI · Questions have been asked about the funding of the CJP.
  
```

```

In [2]:
print("=" * 40)
print("🔍 Tool Information")
print("=" * 40)
print(f"Name       : {search_tool.name}")
print(f"Description  : {search_tool.description}")
print(f"Arguments    : {search_tool.args}")
print("=" * 40)
  
```

```
Out [2]:
=====
🦋 Tool Information
=====
Name      : duckduckgo_search
Description : A wrapper around DuckDuckGo Search. Useful for when you need to answer questions about current events. Input should be a search query.
Arguments  : {'query': {'description': 'search query to look up', 'title': 'Query', 'type': 'string'}}
=====
```

Python REPL

Definition: Execute Python code the model writes and return the output.

Why it exists: Exact math, data manipulation, plotting — things the model shouldn't *guess*.

When developers use it: Data analysis agents, precise calculation.

```
In [3]:
from langchain_experimental.tools import PythonREPLTool
py = PythonREPLTool()
result = py.invoke("print(sum(range(10)))")
print(result)
```

```
Out [3]:
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_25488\1551849229.py:1: DeprecationWarning: `langchain-experimental` is being sunset and is no longer actively maintained. See https://github.com/langchain-ai/langchain-experimental/issues/87 for details.
  from langchain_experimental.tools import PythonREPLTool
Python REPL can execute arbitrary code. Use with caution.
45
```

```
In [4]:
print("=" * 40)
print("🦋 Tool Information")
print("=" * 40)
print(f"Name      : {py.name}")
print(f"Description : {py.description}")
print(f"Arguments  : {py.args}")
print("=" * 40)
```

```
Out [4]:
=====
🦋 Tool Information
=====
Name      : Python_REPL
Description : A Python shell. Use this to execute python commands. Input should be a valid python command. If you want to see the output of a value, you should print it out with `print(...)` .
Arguments  : {'query': {'title': 'Query', 'type': 'string'}}
=====
```

Wikipedia

Definition: Look up and summarize Wikipedia articles.

Why it exists: Reliable encyclopedic facts on demand.

```
In [5]:
import wikipedia
from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper

wikipedia.set_user_agent("MyLangChainApp/1.0 (sachin.kapoor2055@gmail.com)")

# Create the Wikipedia API wrapper
api_wrapper = WikipediaAPIWrapper(top_k_results=1,
                                   doc_content_chars_max=500)

# Create the tool
wikipedia = WikipediaQueryRun(api_wrapper=api_wrapper)
```

```
# Query Wikipedia
result = wikipedia.invoke("who is the president of the United States?")

print(result)
```

Out [5]:

Page: List of United States presidential candidates
 Summary: This article is a list of United States presidential candidates. The first U.S. presidential election was held in 1788–1789, followed by the second in 1792. Presidential elections have been held every four years thereafter. Presidential candidates win the election by winning a majority of the electoral vote. If no candidate wins a majority of the electoral vote, the winner is determined through a contingent election held in the United

```
In [6]:
print("=" * 40)
print("🔧 Tool Information")
print("=" * 40)
print(f"Name      : {wikipedia.name}")
print(f"Description : {wikipedia.description}")
print(f"Arguments  : {wikipedia.args}")
print("=" * 40)
```

Out [6]:

```
=====
🔧 Tool Information
=====
Name      : wikipedia
Description : A wrapper around Wikipedia. Useful for when you need to answer general questions about people, places, companies, facts, historical events, or other subjects. Input should be a search query.
Arguments  : {'query': {'description': 'query to look up on wikipedia', 'title': 'Query', 'type': 'string'}}
```

```
In [7]:
import json
from pprint import pprint
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_experimental.tools import PythonREPLTool
from langchain_community.tools import WikipediaQueryRun
from langchain_community.utilities import WikipediaAPIWrapper
from langchain_ollama import ChatOllama
```

```
# 1. DuckDuckGo
search_tool = DuckDuckGoSearchRun()

# 2. Python REPL
python_tool = PythonREPLTool()

# 3. Wikipedia
api_wrapper = WikipediaAPIWrapper(top_k_results=1,
                                   doc_content_chars_max=500)

wikipedia_tool = WikipediaQueryRun(api_wrapper=api_wrapper)

# Toolkit
tools = [search_tool, python_tool, wikipedia_tool]
```

Out [7]:

d:\Dev_Workspace\LangChain\langchainenv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
 from .autonotebook import tqdm as notebook_tqdm

```
In [8]:
print(tools)
```

Out [8]:

```
[DuckDuckGoSearchRun(api_wrapper=DuckDuckGoSearchAPIWrapper(region='wt-wt', safesearch='moderate', time='y', max_results=5, backend='auto', source='text')), PythonREPLTool(python_repl=PythonREPL(globals={'__name__': 'langchain_experimental.tools.python.tool', '__doc__': 'A tool for running python code in a REPL.', '__package__': 'langchain_experimental.tools.python', '__loader__': <frozen_importlib_external.SourceFileLoader object at 0x0000027ADBE30F50>,
```

```
'__spec__': ModuleSpec(name='langchain_experimental.tools.python.tool', loader=<_frozen_importlib_external.SourceFileLoader object at 0x0000027ADBE30F50>, origin='d:\\Dev_Workspace\\LangChain\\langchainenv\\Lib\\site-packages\\langchain_experimental\\tools\\python\\tool.py'), '__file__': 'd:\\Dev_Workspace\\LangChain\\langchainenv\\Lib\\site-packages\\langchain_experimental\\tools\\python\\tool.py', '__cached__': 'd:\\Dev_Workspace\\LangChain\\langchainenv\\Lib\\site-packages\\langchain_experimental\\tools\\python\\__pycache__\\tool.cpython-313.pyc', '__builtins__': {'__name__': 'builtins', '__doc__': "Built-in functions, types, exceptions, and other objects.\\n\\nThis module provides direct access to all 'built-in' identifiers of Python; for example, builtins.len is\\nthe full name for the built-in function len().\\n\\nThis module is not normally accessed explicitly by most\\napplications, but can be useful in modules that provide\\nobjects with the same name as a built-in value, but in\\nwhich the built-in of that name is also needed.", '__package__': '', '__loader__': <class '_frozen_importlib.BuiltinImporter'>, origin='built-in'), '__build_class__': <built-in function __build_class__>, '__import__': <built-in function __import__>, 'abs': <built-in function abs>, 'all': <built-in function all>, 'any': <built-in function any>, 'ascii': <built-in function ascii>, 'bin': <built-in function bin>, 'breakpoint': <built-in function breakpoint>, 'callable': <built-in function callable>, 'chr': <built-in function chr>, 'compile': <built-in function compile>, 'delattr': <built-in function delattr>, 'dir': <built-in function dir>, 'divmod': <built-in function divmod>, 'eval': <built-in function eval>, 'exec': <built-in function exec>, 'format': <built-in function format>, 'getattr': <built-in function getattr>, 'globals': <built-in function globals>, 'hasattr': <built-in function hasattr>, 'hash': <built-in function hash>, 'hex': <built-in function hex>, 'id': <built-in function id>, 'input': <bound method Kernel.raw_input of <ipykernel.ipkernel.IPythonKernel object at 0x0000027AD9082660>>, 'isinstance': <built-in function isinstance>, 'issubclass': <built-in function issubclass>, 'iter': <built-in function iter>, 'iter': <built-in function iter>, 'len': <built-in function len>, 'locals': <built-in function locals>, 'max': <built-in function max>, 'min': <built-in function min>, 'next': <built-in function next>, 'nnext': <built-in function anext>, 'oct': <built-in function oct>, 'ord': <built-in function ord>, 'pow': <built-in function pow>, 'print': <built-in function print>, 'repr': <built-in function repr>, 'round': <built-in function round>, 'setattr': <built-in function setattr>, 'sorted': <built-in function sorted>, 'sum': <built-in function sum>, 'vars': <built-in function vars>, 'None': None, 'Ellipsis': Ellipsis, 'NotImplemented': NotImplemented, 'False': False, 'True': True, 'bool': <class 'bool'>, 'memoryview': <class 'memoryview'>, 'bytearray': <class 'bytearray'>, 'bytes': <class 'bytes'>, 'classmethod': <class 'classmethod'>, 'complex': <class 'complex'>, 'dict': <class 'dict'>, 'enumerate': <class 'enumerate'>, 'filter': <class 'filter'>, 'float': <class 'float'>, 'frozenset': <class 'frozenset'>, 'property': <class 'property'>, 'int': <class 'int'>, 'list': <class 'list'>, 'map': <class 'map'>, 'object': <class 'object'>, 'range': <class 'range'>, 'reversed': <class 'reversed'>, 'set': <class 'set'>, 'slice': <class 'slice'>, 'staticmethod': <class 'staticmethod'>, 'str': <class 'str'>, 'super': <class 'super'>, 'tuple': <class 'tuple'>, 'type': <class 'type'>, 'zip': <class 'zip'>, '__debug__': True, 'BaseException': <class 'BaseException'>, 'BaseExceptionGroup': <class 'BaseExceptionGroup'>, 'Exception': <class 'Exception'>, 'GeneratorExit': <class 'GeneratorExit'>, 'KeyboardInterrupt': <class 'KeyboardInterrupt'>, 'SystemExit': <class 'SystemExit'>, 'ArithmeticError': <class 'ArithmeticError'>, 'AssertionError': <class 'AssertionError'>, 'AttributeError': <class 'AttributeError'>, 'BufferError': <class 'BufferError'>, 'EOFError': <class 'EOFError'>, 'ImportError': <class 'ImportError'>, 'LookupError': <class 'LookupError'>, 'MemoryError': <class 'MemoryError'>, 'NameError': <class 'NameError'>, 'OSError': <class 'OSError'>, 'ReferenceError': <class 'ReferenceError'>, 'RuntimeError': <class 'RuntimeError'>, 'StopAsyncIteration': <class 'StopAsyncIteration'>, 'StopIteration': <class 'StopIteration'>, 'SyntaxError': <class 'SyntaxError'>, 'SystemError': <class 'SystemError'>, 'TypeError': <class 'TypeError'>, 'ValueError': <class 'ValueError'>, 'Warning': <class 'Warning'>, 'FloatingPointError': <class 'FloatingPointError'>, 'OverflowError': <class 'OverflowError'>, 'ZeroDivisionError': <class 'ZeroDivisionError'>, 'BytesWarning': <class 'BytesWarning'>, 'DeprecationWarning': <class 'DeprecationWarning'>, 'EncodingWarning': <class 'EncodingWarning'>, 'FutureWarning': <class 'FutureWarning'>, 'ImportWarning': <class 'ImportWarning'>, 'PendingDeprecationWarning': <class 'PendingDeprecationWarning'>, 'ResourceWarning': <class 'ResourceWarning'>, 'RuntimeWarning': <class 'RuntimeWarning'>, 'SyntaxWarning': <class 'SyntaxWarning'>, 'UnicodeWarning': <class 'UnicodeWarning'>, 'UserWarning': <class 'UserWarning'>, 'BlockingIOError': <class 'BlockingIOError'>, 'ChildProcessError': <class 'ChildProcessError'>, 'ConnectionError': <class 'ConnectionError'>, 'FileExistsError': <class 'FileExistsError'>, 'FileNotFoundError': <class 'FileNotFoundError'>, 'InterruptedError': <class 'InterruptedError'>, 'IsADirectoryError': <class 'IsADirectoryError'>, 'NotADirectoryError': <class 'NotADirectoryError'>, 'PermissionError': <class 'PermissionError'>, 'ProcessLookupError': <class 'ProcessLookupError'>, 'TimeoutError': <class 'TimeoutError'>, 'IndentationError': <class 'IndentationError'>, '_IncompleteInputError': <class '_IncompleteInputError'>, 'IndexError': <class 'IndexError'>, 'KeyError': <class 'KeyError'>, 'ModuleNotFoundError': <class 'ModuleNotFoundError'>, 'NotImplementedError': <class 'NotImplementedError'>, 'PythonFinalizationError': <class 'PythonFinalizationError'>, 'RecursionError': <class 'RecursionError'>, 'UnboundLocalError': <class 'UnboundLocalError'>, 'UnicodeError': <class 'UnicodeError'>, 'BrokenPipeError': <class 'BrokenPipeError'>, 'ConnectionAbortedError': <class 'ConnectionAbortedError'>, 'ConnectionRefusedError': <class 'ConnectionRefusedError'>, 'ConnectionResetError': <class 'ConnectionResetError'>, 'TabError': <class 'TabError'>, 'UnicodeDecodeError': <class 'UnicodeDecodeError'>, 'UnicodeEncodeError': <class 'UnicodeEncodeError'>, 'UnicodeTranslateError': <class 'UnicodeTranslateError'>, 'ExceptionGroup': <class 'ExceptionGroup'>, 'EnvironmentError': <class 'OSError'>, 'IOError': <class 'OSError'>, 'WindowsError': <class 'OSError'>, 'open': <built-in function open>, 'copyright': Copyright (c) 2001-2024 Python Software Foundation.
```

All Rights Reserved.

Copyright (c) 2000 BeOpen.com.

All Rights Reserved.

Copyright (c) 1995-2001 Corporation for National Research Initiatives.

All Rights Reserved.

Copyright (c) 1991-1995 Stichting Mathematisch Centrum, Amsterdam.

All Rights Reserved., 'credits': Thanks to CWI, CNRI, BeOpen, Zope Corporation, the Python Software Foundation, and a cast of thousands for supporting Python

development. See www.python.org for more information., 'license': Type license() to see the full license text, 'help': Type help() for interactive help, or help(object) for help about object., 'execfile': <function execfile at 0x0000027AD9177E20>, 'runfile': <function runfile at 0x0000027AD92DC180>, '__IPYTHON__': True, 'display': <function display at 0x0000027AD62E4EA0>, 'get_ipython': <bound method InteractiveShell.get_ipython of <ipykernel.zmqshell.ZMQInteractiveShell object at 0x0000027AD93DC590>>}, 'ast': <module 'ast' from 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qbz5n2kfra8p0\\Lib\\ast.py'>, 're': <module 're' from 'C:\\Program Files\\WindowsApps\\PythonSoftwareFoundation.Python.3.13_3.13.3824.0_x64__qbz5n2kfra8p0\\Lib\\re__init__.py'>, 'sys': <module 'sys' (built-in)>, 'redirect_stdout': <class 'contextlib.redirect_stdout'>, 'StringIO': <class '_io.StringIO'>, 'Any': typing.Any, 'Dict': typing.Dict, 'Optional': typing.Optional, 'Type': typing.Type, 'AsyncCallbackManagerForToolRun': <class 'langchain_core.callbacks.manager.AsyncCallbackManagerForToolRun'>, 'CallbackManagerForToolRun': <class 'langchain_core.callbacks.manager.CallbackManagerForToolRun'>, 'run_in_executor': <function run_in_executor at 0x0000027AD9D74720>,

```
'BaseTool': <class 'langchain_core.tools.base.BaseTool'>, 'BaseModel': <class 'pydantic.main.BaseModel'>, 'Field': <function Field at 0x0000027AD9831F80>, 'model_validator': <function model_validator at 0x0000027AD9ACE200>, 'PythonREPL': <class 'langchain_experimental.utilities.python.PythonREPL'>, '_get_default_python_repl': <function _get_default_python_repl at 0x0000027ADBE2E660>, 'sanitize_input': <function sanitize_input at 0x0000027ADBE2E700>, 'PythonREPLTool': <class 'langchain_experimental.tools.python.tool.PythonREPLTool'>, 'PythonInputs': <class 'langchain_experimental.tools.python.tool.PythonInputs'>, 'PythonAstREPLTool': <class 'langchain_experimental.tools.python.tool.PythonAstREPLTool'>}, locals=None)),
WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper(wiki_client=<module 'wikipedia' from 'd:\Dev_Workspace\LangChain\langchainenv\Lib\site-packages\wikipedia\__init__.py'>, top_k_results=1, lang='en', load_all_available_meta=False, doc_content_chars_max=500))]
```

In [9]:

```
# Initialize the language model
llm = ChatOllama(model="qwen3:8b")
llm_with_tools = llm.bind_tools(tools)
```

In [10]:

```
for tool in llm_with_tools.kwarg["tools"]:
    function = tool["function"]

    print("=" * 60)
    print(f"🔧 Tool: {function['name']}")
    print("=" * 60)
    print(f"Description:\n{function['description']}")
    print("\nParameters:")

    print(json.dumps(
        function["parameters"],
        indent=4
    ))

    print()
```

Out [10]:

```
=====
🔧 Tool: duckduckgo_search
=====
```

Description:

A wrapper around DuckDuckGo Search. Useful for when you need to answer questions about current events. Input should be a search query.

Parameters:

```
{
  "properties": {
    "query": {
      "description": "search query to look up",
      "type": "string"
    }
  },
  "required": [
    "query"
  ],
  "type": "object"
}
```

```
=====
🔧 Tool: Python_REPL
=====
```

Description:

A Python shell. Use this to execute python commands. Input should be a valid python command. If you want to see the output of a value, you should print it out with `print(...)`.

Parameters:

```
{
  "properties": {
    "query": {
      "type": "string"
    }
  },
  "required": [
    "query"
  ],
  "type": "object"
}
```

```
=====
```

```
🔧 Tool: wikipedia
```

```
=====
```

Description:

A wrapper around Wikipedia. Useful for when you need to answer general questions about people, places, companies, facts, historical events, or other subjects. Input should be a search query.

Parameters:

```
{
  "properties": {
    "query": {
      "description": "query to look up on wikipedia",
      "type": "string"
    }
  },
  "required": [
    "query"
  ],
  "type": "object"
}
```

```
In [11]:
```

```
# This will print all the accessible tools to LLM with their respective names, descriptions, and parameters in a structured format
pprint(llm_with_tools.kwargs)
```

```
Out [11]:
```

```
{'tools': [{'function': {'description': 'A wrapper around DuckDuckGo Search. '
                                'Useful for when you need to answer '
                                'questions about current events. Input '
                                'should be a search query.',
                                'name': 'duckduckgo_search',
                                'parameters': {'properties': {'query': {'description': 'search '
                                                'query '
                                                'to '
                                                'look '
                                                'up',
                                                'type': 'string'}},
                                                'required': ['query'],
                                                'type': 'object'}},
                                'type': 'function'},
            {'function': {'description': 'A Python shell. Use this to execute '
                                'python commands. Input should be a '
                                'valid python command. If you want to '
                                'see the output of a value, you should '
                                'print it out with `print(...)`.',
                                'name': 'Python_REPL',
                                'parameters': {'properties': {'query': {'type': 'string'}},
                                                'required': ['query'],
                                                'type': 'object'}},
                                'type': 'function'},
            {'function': {'description': 'A wrapper around Wikipedia. Useful '
                                'for when you need to answer general '
                                'questions about people, places, '
                                'companies, facts, historical events, '
                                'or other subjects. Input should be a '
                                'search query.',
                                'name': 'wikipedia',
                                'parameters': {'properties': {'query': {'description': 'query '
                                                'to '
                                                'look '
                                                'up '
                                                'on '
                                                'wikipedia',
                                                'type': 'string'}},
                                                'required': ['query'],
                                                'type': 'object'}},
                                'type': 'function'}}]}
```

```
In [12]:
```

```
response = llm_with_tools.invoke("Who is the current president of the United States?")
print(response)
```

```
Out [12]:
```

```
content='' additional_kwargs={} response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:22:19.3339521Z', 'done': True,
'done_reason': 'stop', 'total_duration': 26997232500, 'load_duration': 3329600, 'prompt_eval_count': 323, 'prompt_eval_duration':
563556000, 'eval_count': 361, 'eval_duration': 26382668000, 'logprobs': None, 'model_name': 'qwen3:8b', 'model_provider': 'ollama'}
id='lc_run--01a0b99d-a84f-7070-9f70-36d7b3c33ba2-0' tool_calls=[{'name': 'duckduckgo_search', 'args': {'query': 'current president of
the United States'}, 'id': 'b7d4554c-105b-458a-9ab9-1a1fc0e71e7a', 'type': 'tool_call'}] invalid_tool_calls=[] usage_metadata=
{'input_tokens': 323, 'output_tokens': 361, 'total_tokens': 684}
```

In [13]:

```
print(response.tool_calls)
```

Out [13]:

```
[{'name': 'duckduckgo_search', 'args': {'query': 'current president of the United States'}, 'id': 'b7d4554c-105b-458a-9ab9-
1a1fc0e71e7a', 'type': 'tool_call'}]
```

In [14]:

```
# Execute the requested tool
tool_call = response.tool_calls[0]
tool_name = tool_call["name"]
tool_args = tool_call["args"]
```

In [15]:

```
print("LLM requested:")
print(response.tool_calls)

# Execute the requested tool
tool_call = response.tool_calls[0]

tool_name = tool_call["name"]
tool_args = tool_call["args"]

for tool in tools:
    if tool.name == tool_name:
        tool_result = tool.invoke(tool_args)
        break

print("\nTool result:")
print(tool_result)
```

Out [15]:

```
LLM requested:
[{'name': 'duckduckgo_search', 'args': {'query': 'current president of the United States'}, 'id': 'b7d4554c-105b-458a-9ab9-
1a1fc0e71e7a', 'type': 'tool_call'}]
```

Tool result:

1 day ago - In all, 45 individuals have served, ... as of the current 60th four-year term. Donald Trump is the 47th and current president since January 20, 2025. In July 1776, the Thirteen Colonies, represented at the Second Continental Congress in Philadelphia, unanimously adopted the United States Declaration ... 4 days ago - "Presidential and Vice Presidential ... and Current Legislation" (PDF). Congressional Research Service. Retrieved November 14, 2023. Peabody, Bruce G.; Gant, Scott E. (1999). "The Twice and Future President: Constitutional Interstices and the Twenty-Second Amendment". Minnesota Law Review. 83 (565). ISSN 0026-5535. Rossiter, Clinton (1962). "Powers of the United States President ... 2 hours ago - As president, Trump described illegal immigration as an "invasion" of the United States and drastically escalated immigration enforcement. He implemented harsh policies against asylum seekers and deployed nearly 6,000 troops to the U.S.-Mexico border to stop illegal crossings. 2 days ago - As president, Biden signed the American Rescue Plan Act in response to the COVID-19 pandemic and recession. He signed bipartisan bills on infrastructure and manufacturing. Biden proposed the Build Back Better Act, part of which was incorporated into the Inflation Reduction Act passed in 2022. He appointed Ketanji Brown Jackson to the Supreme Court of the United States... 2 days ago - Donald Trump's second and current tenure as the president of the United States began upon his inauguration as the 47th president on January 20, 2025. Trump, a Republican, previously served as the 45th president from 2017 to 2021. He lost re-election to Democratic nominee Joe Biden in 2020, ...

In [16]:

```
response = llm_with_tools.invoke("print(sum(range(10)))")
print(response.tool_calls)
```

Out [16]:

```
[{'name': 'Python_REPL', 'args': {'query': 'print(sum(range(10)))'}, 'id': '6d079c5a-930c-4b6c-aa3b-baf7ea8a9abd', 'type':
'tool_call'}]
```

In [17]:

```
print("LLM requested:")
print(response.tool_calls)

# Execute the requested tool
tool_call = response.tool_calls[0]

tool_name = tool_call["name"]
tool_args = tool_call["args"]

for tool in tools:
    if tool.name == tool_name:
        tool_result = tool.invoke(tool_args)
        break

print("\nTool result:")
print(tool_result)
```

Out [17]:

LLM requested:

```
[{'name': 'Python_REPL', 'args': {'query': 'print(sum(range(10)))'}, 'id': '6d079c5a-930c-4b6c-aa3b-baf7ea8a9abd', 'type': 'tool_call'}]
```

Tool result:

45

3. tool decorator

19 de septiembre de 2026



Contenido

1. The @tool Decorator

1. Simple Definition

2. Why Does It Exist?

Before (manual schema)

...and you still have to wire the actual function separately. Two things to keep in sync.

After (@tool)

name, description, and schema are ALL derived from the function automatically.

3. Real-Life Analogy

4. Where It Fits in LangChain Architecture

5. Internal Working

6. Attributes / Options

the docstring (→ description)

type hints (→ args_schema)

schema: origin:str, destination:str, passengers:int (all required)

name (override)

args_schema (Pydantic, for richer arguments)

return_direct

response_format="content_and_artifact"

Async tools

Works with .ainvoke() and async agents.

1. The @tool Decorator

The **easiest and most common** way to create a tool: write a normal Python function, add `@tool` above it, and it becomes a full LangChain tool the model can call.

1. Simple Definition

Kid version: You already know how to write a little Python function. Putting `@tool` on top is like sticking a **name tag** on it that says "Hi model, you're allowed to use me, and here's what I do." Now the AI can pick it up and use it.

Professional definition: `@tool` is a decorator that converts a Python function into a `StructuredTool`. It automatically derives the tool's **name** (function name), **description** (docstring), and **argument schema** (type hints) so the model knows how to call it.

```
from langchain_core.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers together."""
    return a * b

print(multiply.name)          # 'multiply'
print(multiply.description)   # 'Multiply two numbers together.'
print(multiply.args)         # {'a': {'type': 'integer', ...}, 'b': {...}}
print(multiply.invoke({"a": 3, "b": 4})) # 12
```

2. Why Does It Exist?

The problem: For the model to call a function, it needs a machine-readable **schema**: the tool's name, what it does, and its arguments with types. Writing that JSON schema by hand for every function is tedious and easy to get out of sync with the code.

Before (manual schema)

```
weather_schema = {
    "name": "get_weather",
    "description": "Get weather for a city",
    "parameters": {
        "type": "object",
        "properties": {"city": {"type": "string", "description": "the city"}},
        "required": ["city"],
    },
}

# ...and you still have to wire the actual function separately. Two things to keep in sync.
```

After (`@tool`)

```
@tool
def get_weather(city: str) -> str:
    """Get weather for a city."""
    return f"18°C in {city}"

# name, description, and schema are ALL derived from the function automatically.
```

One source of truth (the function) → the schema is generated from your **type hints** and **docstring**. Less code, no drift.

3. Real-Life Analogy

Putting a label on a machine in a workshop 🛠️. The machine already works; you just add a clear label ("Drill — makes holes, needs: material, hole size") so anyone (the model) knows what it does and what inputs it needs, without reading the internal wiring.

4. Where It Fits in LangChain Architecture

```
BaseTool
|
▼
StructuredTool    ← what @tool actually produces
▲
| created by
@tool (decorator over your function)
```

- `@tool` is a **shortcut** that builds a `StructuredTool` from your function.
- The resulting object has everything the model needs: `name`, `description`, `args_schema`, and an `.invoke()` method.

5. Internal Working

```
@tool
def get_weather(city: str) -> str:
    """Get weather for a city."""
    ...
    |
    ▼
INSPECT the function:
- name      ← function name → "get_weather"
- description ← docstring    → "Get weather for a city."
- args_schema ← type hints   → {city: string, required}
    |
    ▼
WRAP into a StructuredTool object with an .invoke() that calls your function
    |
```

▼
Ready to bind_tools([...]) to a model (file 5)

The docstring and type hints aren't just documentation — they are **sent to the model** and directly affect how well it uses the tool.

6. Attributes / Options

the docstring (→ description)

Definition: The function's docstring becomes the tool's `description` — the model reads this to decide *when* to use the tool.

Why it exists: The description is the model's main clue about the tool's purpose.

When developers use it: Always — write a clear, specific docstring.

Real-life use case: The instruction label on the machine.

```
@tool
def search_docs(query: str) -> str:
    """Search the company knowledge base for a query and return the top matching passage.
    Use this when the user asks about internal policies or product details."""
    ...
```

💡 A vague docstring = the model misuses or ignores the tool. Be specific about what it does and when to use it.

type hints (→ args_schema)

Definition: Parameter type hints define the argument schema and types the model must provide.

Why it exists: The model needs to know each argument's name and type.

Real-life use case: The list of inputs the machine needs.

```
@tool
def book_flight(origin: str, destination: str, passengers: int) -> str:
    """Book a flight."""
    ...

# schema: origin:str, destination:str, passengers:int (all required)
```

name (override)

Definition: By default the tool name = function name; you can override it.

Why it exists: Sometimes you want a cleaner/clearer tool name than the Python function name.

```
@tool("web_search")          # custom name
def _search_impl(query: str) -> str:
    """Search the web."""
    ...
```

args_schema (Pydantic, for richer arguments)

Definition: Pass a Pydantic model to precisely control argument descriptions/validation.

Why it exists: Per-argument descriptions dramatically improve how the model fills them.

When developers use it: When arguments need explanations or validation.

```
from pydantic import BaseModel, Field

class SearchInput(BaseModel):
    query: str = Field(description="the search query, phrased as keywords")
    top_k: int = Field(default=5, description="how many results to return")

@tool(args_schema=SearchInput)
```

```
def web_search(query: str, top_k: int = 5) -> str:
    """Search the web."""
    ...
```

return_direct

Definition: If `True`, an agent returns this tool's output **directly** to the user instead of looping back to the model.

Why it exists: For tools whose raw output is the final answer (saves a round-trip and tokens).

Internal working: (return_direct=False)

```
User
↓
LLM
↓
multiply tool
↓
100
↓
LLM
↓
Final answer
```

The tool result goes back to the LLM, and the LLM gets a chance to explain or continue.

Internal working: (return_direct=True)

```
User
↓
LLM
↓
multiply tool
↓
100
↓
LLM
↓
Final answer
```

The agent stops after the tool and returns the tool's result directly to the user instead of sending it back through the LLM.

```
@tool(return_direct=True)
def get_balance(account_id: str) -> str:
    """Return the account balance."""
    ...
```

response_format="content_and_artifact"

Definition: Lets a tool return both a **string for the model** and a raw **artifact** for your code (e.g. a dataframe, image bytes) that the model shouldn't see in full.

Why it exists: Some tool outputs are huge/binary; the model only needs a summary.

```
@tool(response_format="content_and_artifact")
def run_query(sql: str) -> tuple[str, list]:
    """Run a SQL query."""
    rows = [...]
    return f"Returned {len(rows)} rows.", rows # (content_for_model, artifact_for_code)
```

Async tools

```
@tool
async def fetch_url(url: str) -> str:
```

```

"""Fetch a URL asynchronously."""
async with httpx.AsyncClient() as client:
    return (await client.get(url)).text
# Works with .ainvoke() and async agents.

```

In [2]:

```

from langchain_core.tools import tool
from langchain_ollama import ChatOllama

# Initialize the Language model
llm = ChatOllama(model="qwen3:8b")

# step 1: define the tool
@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

```

In [3]:

```

print("=" * 40)
print("🔧 Tool Information")
print("=" * 40)
print(f"Name      : {multiply.name}")
print(f"Description : {multiply.description}")
print(f"Arguments   : {multiply.args}")
print("=" * 40)

```

Out [3]:

```

=====
🔧 Tool Information
=====
Name      : multiply
Description : Multiply two numbers.
Arguments   : {'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B', 'type': 'integer'}}
=====

```

In [4]:

```
multiply.args_schema.model_json_schema()
```

Out [4]:

```

{'description': 'Multiply two numbers.',
 'properties': {'a': {'title': 'A', 'type': 'integer'},
               'b': {'title': 'B', 'type': 'integer'}},
 'required': ['a', 'b'],
 'title': 'multiply',
 'type': 'object'}

```

In [5]:

```
multiply.invoke({"a": 5, "b": 3})
```

Out [5]:

15

In [6]:

```

#step2: bind the tool to the Language model
llm_with_tools = llm.bind_tools([multiply])

```

In [7]:

```

# step3: tool calling
msg = llm_with_tools.invoke("What's 12 times 7?")
msg

```

Out [7]:

```

AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:14:42.1158616Z',
'done': True, 'done_reason': 'stop', 'total_duration': 12723661000, 'load_duration': 6031925800, 'prompt_eval_count': 143,
'prompt_eval_duration': 418356000, 'eval_count': 97, 'eval_duration': 6268031000, 'logprobs': None, 'model_name': 'qwen3:8b',
'model_provider': 'ollama'}, id='lc_run--01a0b996-e60e-7e23-a3e1-b22666f0f349-0', tool_calls=[{'name': 'multiply', 'args': {'a': 12,

```

```
'b': 7}, 'id': '61033cec-002d-4c8f-9f97-f35ac87dd64a', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 143, 'output_tokens': 97, 'total_tokens': 240})
```

In [8]:

```
msg.tool_calls
```

Out [8]:

```
[{'name': 'multiply',
  'args': {'a': 12, 'b': 7},
  'id': '61033cec-002d-4c8f-9f97-f35ac87dd64a',
  'type': 'tool_call'}]
```

In [9]:

```
# step 4: tool execution
```

```
result = multiply.invoke(llm_with_tools.invoke("can you multiply 3 with 10").tool_calls[0]['args'])
result
```

Out [9]:

```
30
```

In [10]:

```
from langchain_core.tools import tool
from langchain_ollama import ChatOllama
```

```
# step 1: define the tool
```

```
@tool
def add(a: float, b: float) -> float:
    """Add two numbers."""
    return a + b
```

```
@tool
def subtract(a: float, b: float) -> float:
    """Subtract b from a."""
    return a - b
```

```
@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b
```

```
@tool
def divide(a: float, b: float) -> float:
    """Divide a by b."""
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

```
# Create toolkit
```

```
tools = [add, subtract, multiply, divide]
```

```
llm = ChatOllama(model="qwen3:8b")
```

```
#step2: bind the tool to the language model
```

```
llm_with_tools = llm.bind_tools(tools)
```

```
# -----
```

```
# Check available tools
```

```
# -----
```

```
for tool in tools:
    print("=" * 40)
    print(f"Tool: {tool.name}")
    print(f>Description: {tool.description}")
    print(f"Schema: {tool.args}")
    print(f"Schema JSON: {tool.args_schema.model_json_schema()}")
    print("=" * 40)
```

Out [10]:

```
=====
Tool: add
```

```

Description: Add two numbers.
Schema: {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}
Schema JSON: {'description': 'Add two numbers.', 'properties': {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'add', 'type': 'object'}
=====
Tool: subtract
Description: Subtract b from a.
Schema: {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}
Schema JSON: {'description': 'Subtract b from a.', 'properties': {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'subtract', 'type': 'object'}
=====
Tool: multiply
Description: Multiply two numbers.
Schema: {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}
Schema JSON: {'description': 'Multiply two numbers.', 'properties': {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'multiply', 'type': 'object'}
=====
Tool: divide
Description: Divide a by b.
Schema: {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}
Schema JSON: {'description': 'Divide a by b.', 'properties': {'a': {'title': 'A', 'type': 'number'}, 'b': {'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'divide', 'type': 'object'}
=====

```

```

In [11]:
# step3: tool calling
response = llm_with_tools.invoke("What is 25 multiplied by 12?")
response

```

```

Out [11]:
AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:15:03.9123362Z', 'done': True, 'done_reason': 'stop', 'total_duration': 11965512400, 'load_duration': 2055100, 'prompt_eval_count': 273, 'prompt_eval_duration': 422968000, 'eval_count': 165, 'eval_duration': 11526962000, 'logprobs': None, 'model_name': 'qwen3:8b', 'model_provider': 'ollama'}, id='lc_run--01a0b997-3e2a-7142-b1c5-a64530135edf-0', tool_calls=[{'name': 'multiply', 'args': {'b': 12, 'a': 25}, 'id': '6169897b-4064-4759-9668-aa7bf6cabb09', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 273, 'output_tokens': 165, 'total_tokens': 438})

```

```

In [12]:
response.tool_calls

```

```

Out [12]:
[{'name': 'multiply',
  'args': {'b': 12, 'a': 25},
  'id': '6169897b-4064-4759-9668-aa7bf6cabb09',
  'type': 'tool_call'}]

```

```

In [13]:
print("LLM requested:")
print(response.tool_calls)

# Execute the requested tool
tool_call = response.tool_calls[0]

tool_name = tool_call["name"]
tool_args = tool_call["args"]

for tool in tools:
    if tool.name == tool_name:
        tool_result = tool.invoke(tool_args)
        break

print("\nTool result:")
print(tool_result)

```

```

Out [13]:
LLM requested:
[{'name': 'multiply', 'args': {'b': 12, 'a': 25}, 'id': '6169897b-4064-4759-9668-aa7bf6cabb09', 'type': 'tool_call'}]

```

Tool result:
300.0

4. structured tool

19 de septiembre de 2026



Contenido

2. StructuredTool & Tool

1. Simple Definition

2. Why Do They Exist?

Before (can't decorate a function you don't own)

@tool ← you can't add this to some_library.search

After (StructuredTool.from_function)

3. Real-Life Analogy

4. Where They Fit in LangChain Architecture

5. Internal Working

6. StructuredTool — key parameters

func / coroutine

name / description

args_schema

return_direct

7. The older Tool class (single string input)

Building tools dynamically (a real advantage)

tools built programmatically — impossible with a static @tool decorator.

Which creation method should I use?

2. StructuredTool & Tool

The **non-decorator** ways to create tools. Use these when you want to build a tool from an existing function at runtime, wrap a function you don't own, or configure the tool explicitly.

1. Simple Definition

Kid version: In `tool_decorator` notebooks stuck a label on a function with `@tool`. Sometimes you can't put a sticker on the function directly (maybe it's someone else's function, or you're deciding at runtime). So instead you **build the labeled tool by hand**, passing in the function and its label separately.

Professional definition:

- `StructuredTool.from_function(...)` builds a tool from a function with a **multi-argument** schema (the modern, recommended constructor).
- `Tool(...)` is the older, simpler class for tools that take a **single string input** (common in classic agents).

```
from langchain_core.tools import StructuredTool

def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

calculator = StructuredTool.from_function(
    func=multiply,
    name="calculator",
    description="Multiply two integers together.",
)

calculator.invoke({"a": 6, "b": 7})  # 42
```

2. Why Do They Exist?

The problem: `@tool` is great when *you* are writing the function and can decorate it. But sometimes:

- The function is **third-party** (you can't add a decorator to it).
- You're creating tools **dynamically** (in a loop, from config).
- You want to attach **separate sync and async** implementations.
- You want to set every field **explicitly** rather than infer it.

Before (can't decorate a function you don't own)

```
import some_library
# @tool ← you can't add this to some_library.search
```

After (`StructuredTool.from_function`)

```
search_tool = StructuredTool.from_function(
    func=some_library.search,
    name="search",
    description="Search the external service for a query.",
)
```

They give you a **programmatic** way to build tools, decoupled from the function definition.

3. Real-Life Analogy

Buying a labeled tool vs. labeling your own 🛠️ . `@tool` is putting a sticker on a tool you made. `StructuredTool.from_function` is going to the store, picking any tool off the shelf (even one someone else made), and attaching your own clear label to it before handing it to a worker.

4. Where They Fit in LangChain Architecture

```
BaseTool
├── StructuredTool ← multi-argument tools (from_function) ← RECOMMENDED
└── Tool ← single-string-input tools (older style)
```

- `@tool` is just a convenience that produces a `StructuredTool`.
- `StructuredTool` supports rich, multi-argument schemas (like `@tool`).
- `Tool` is the legacy single-input form — still seen in older agent code.

5. Internal Working

```
StructuredTool.from_function(func=multiply, name=..., description=..., args_schema=...)
├──
└── ▼
    If args_schema not given → INFER it from func's type hints (like @tool does)
    └──
        ▼
        BUILD a StructuredTool with:
            name, description, args_schema, func (sync), coroutine (async, optional)
            └──
                ▼
                .invoke({...}) → validates args against schema → calls func
```

6. StructuredTool — key parameters

func / coroutine

Definition: The sync function (`func`) and/or async function (`coroutine`) the tool runs.

Why it exists: Lets you supply implementations explicitly, including separate async.

When developers use it: Always provide at least one.

```
StructuredTool.from_function(func=sync_search, coroutine=async_search, name="search",
                             description="Search the web.")
```

name / description

Definition: The tool's identifier and its purpose text (shown to the model).

Why it exists: Same role as with `@tool` — the model reads these to decide when/how to call.

Real-life use case: The label on the shelf tool.

```
StructuredTool.from_function(func=f, name="get_stock_price",
                             description="Get the latest stock price for a ticker symbol.")
```

args_schema

Definition: A Pydantic model precisely defining the arguments (names, types, descriptions, validation).

Why it exists: Full control over what the model must pass; per-arg descriptions boost accuracy.

When developers use it: When you want validation or clear argument docs (recommended).

```
from pydantic import BaseModel, Field

class MultiplyInput(BaseModel):
    a: int = Field(description="the first number")
    b: int = Field(description="the second number")

StructuredTool.from_function(
    func=multiply, name="multiply",
    description="Multiply two numbers.", args_schema=MultiplyInput,
)
```

return_direct

Definition: If `True`, an agent returns the tool's output directly as the final answer.

```
StructuredTool.from_function(func=f, name="lookup", description="...", return_direct=True)
```

7. The older `Tool` class (single string input)

Definition: `Tool` wraps a function that takes **one string** and returns a string.

Why it exists: Classic ReAct-style agents passed a single string to each tool. Still valid, but `StructuredTool` is preferred for anything with multiple/typed arguments.

Real-life use case: A simple "give it a query string, get a string back" utility.

```
from langchain_core.tools import Tool

def search(query: str) -> str:
    return f"results for {query}"

search_tool = Tool(
    name="search",
    description="Search the web. Input should be a search query string.",
    func=search,
)

search_tool.invoke("langchain tools")  # 'results for langchain tools'
```

Rule of thumb: prefer `StructuredTool` / `@tool`. Use `Tool` only when a tool genuinely takes one string, or you're maintaining older code.

Building tools dynamically (a real advantage)

```
from langchain_core.tools import StructuredTool

def make_unit_converter(factor: float, name: str, desc: str):
    def convert(value: float) -> float:
        return value * factor
    return StructuredTool.from_function(func=convert, name=name, description=desc)

tools = [
    make_unit_converter(2.20462, "kg_to_lb", "Convert kilograms to pounds."),
    make_unit_converter(0.621371, "km_to_mi", "Convert kilometers to miles."),
]

# `tools` built programmatically – impossible with a static @tool decorator.
```

Which creation method should I use?

Situation	Use
You're writing the function yourself	<code>@tool</code> — simplest
Function is third-party / built at runtime	<code>StructuredTool.from_function</code>
Need explicit sync + async, or full field control	<code>StructuredTool.from_function</code>
Legacy single-string-input tool	<code>Tool</code>
Need custom state, lifecycle, or overrides	subclass <code>BaseTool</code>

```
In [2]:
from langchain_core.tools import StructuredTool
from pydantic import BaseModel, Field
from langchain_ollama import ChatOllama

# Initialize the language model
llm = ChatOllama(model="qwen3:8b")
```

```
In [3]:
# step 1: define the tool with schema
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")

def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

multiply_tool = StructuredTool.from_function(
    func = multiply,
    name = "multiply",
    description = "multiply two numbers",
    args_schema = MultiplyInput
)
```

```
Out [3]:
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_2328\3256796255.py:3: PydanticDeprecatedSince20: Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    a: int = Field(required=True, description="The first number to add")
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_2328\3256796255.py:4: PydanticDeprecatedSince20: Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
    b: int = Field(required=True, description="The second number to add")
```

```
In [4]:
```

```

print("=" * 40)
print("🔧 Tool Information")
print("=" * 40)
print(f"Name          : {multiply_tool.name}")
print(f"Description : {multiply_tool.description}")
print(f"Arguments   : {multiply_tool.args}")
print("=" * 40)

```

```

Out [4]:
=====
🔧 Tool Information
=====
Name          : multiply
Description : multiply two numbers
Arguments   : {'a': {'description': 'The first number to add', 'title': 'A', 'type': 'integer'}, 'b': {'description': 'The second
number to add', 'title': 'B', 'type': 'integer'}}
=====

```

```

In [5]:
multiply_tool.args_schema.model_json_schema()

```

```

Out [5]:
{'properties': {'a': {'description': 'The first number to add',
  'required': True,
  'title': 'A',
  'type': 'integer'},
  'b': {'description': 'The second number to add',
  'required': True,
  'title': 'B',
  'type': 'integer'}},
 'required': ['a', 'b'],
 'title': 'MultiplyInput',
 'type': 'object'}

```

```

In [6]:
multiply_tool.invoke({"a": 5, "b": 3})

```

```

Out [6]:
15

```

```

In [7]:
#step2: bind the tool to the language model
llm_with_tools = llm.bind_tools([multiply_tool])

```

```

In [8]:
# step3: tool calling
msg = llm_with_tools.invoke("What's 12 times 7?")
msg

```

```

Out [8]:
AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:16:23.0140247Z',
'done': True, 'done_reason': 'stop', 'total_duration': 13671501400, 'load_duration': 3127300, 'prompt_eval_count': 158,
'prompt_eval_duration': 533250000, 'eval_count': 186, 'eval_duration': 13099721000, 'logprobs': None, 'model_name': 'qwen3:8b',
'model_provider': 'ollama'}, id='lc_run--01a0b998-6c78-7f00-bf41-372d4861715c-0', tool_calls=[{'name': 'multiply', 'args': {'a': 12,
'b': 7}, 'id': '21fe9676-fca6-47d1-9f4e-46cb00bd5cfe', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens':
158, 'output_tokens': 186, 'total_tokens': 344})

```

```

In [9]:
msg.tool_calls

```

```

Out [9]:
[{'name': 'multiply',
  'args': {'a': 12, 'b': 7},
  'id': '21fe9676-fca6-47d1-9f4e-46cb00bd5cfe',
  'type': 'tool_call'}]

```

```

In [10]:

```

```
# step 4: tool execution
result = multiply_tool.invoke(llm_with_tools.invoke("can you multiply 3 with 10").tool_calls[0]['args'])
result
```

Out [10]:

30

In [11]:

```
from langchain_core.tools import StructuredTool
from langchain_ollama import ChatOllama
from pydantic import BaseModel, Field
```

STEP 1: Define input schemas

```
class AddInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")
```

```
class SubtractInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")
```

```
class MultiplyInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")
```

```
class DivideInput(BaseModel):
    a: float = Field(description="Numerator")
    b: float = Field(description="Denominator")
```

In [12]:

STEP 2: Define normal Python functions

```
def add(a: float, b: float) -> float:
    """Add two numbers."""
    return a + b
```

```
def subtract(a: float, b: float) -> float:
    """Subtract b from a."""
    return a - b
```

```
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b
```

```
def divide(a: float, b: float) -> float:
    """Divide a by b."""
    if b == 0:
        return "Cannot divide by zero"
    return a / b
```

In [13]:

STEP 3: Create StructuredTools

```
add_tool = StructuredTool.from_function(
    func=add,
    name="add",
    description="Add two numbers.",
    args_schema=AddInput
)
```

```
subtract_tool = StructuredTool.from_function(
    func=subtract,
    name="subtract",
    description="Subtract b from a.",
    args_schema=SubtractInput
)
```

```

)

multiply_tool = StructuredTool.from_function(
    func=multiply,
    name="multiply",
    description="Multiply two numbers.",
    args_schema=MultiplyInput
)

divide_tool = StructuredTool.from_function(
    func=divide,
    name="divide",
    description="Divide a by b.",
    args_schema=DivideInput
)

```

```

In [14]:
tools = [add_tool, subtract_tool, multiply_tool, divide_tool]

llm = ChatOllama(model="qwen3:8b")

```

```

In [15]:
# STEP 4: Bind tools to LLM
llm_with_tools = llm.bind_tools(tools)

```

```

In [16]:
for tool in tools:
    print("=" * 50)
    print(f"Tool      : {tool.name}")
    print(f"Description: {tool.description}")
    print(f"Arguments  : {tool.args}")
    print(f"Schema JSON:")
    print(tool.args_schema.model_json_schema())
    print("=" * 50)

```

```

Out [16]:
=====
Tool      : add
Description: Add two numbers.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'AddInput', 'type': 'object'}
=====
Tool      : subtract
Description: Subtract b from a.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'SubtractInput', 'type': 'object'}
=====
Tool      : multiply
Description: Multiply two numbers.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'MultiplyInput', 'type': 'object'}
=====
Tool      : divide
Description: Divide a by b.
Arguments  : {'a': {'description': 'Numerator', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Denominator', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'Numerator', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Denominator', 'title': 'B',

```

```
'type': 'number'}}, 'required': ['a', 'b'], 'title': 'DivideInput', 'type': 'object'}
=====
```

```
In [17]:
# STEP 5: Invoke the LLM with tools
response = llm_with_tools.invoke("What is 25 multiplied by 12?")
response

Out [17]:
AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:16:45.9977513Z',
'done': True, 'done_reason': 'stop', 'total_duration': 14521228900, 'load_duration': 4648900, 'prompt_eval_count': 313,
'prompt_eval_duration': 453984000, 'eval_count': 204, 'eval_duration': 14051656000, 'logprobs': None, 'model_name': 'qwen3:8b',
'model_provider': 'ollama'}, id='lc_run--01a0b998-c2f3-7ce1-b282-3ce19a35e563-0', tool_calls=[{'name': 'multiply', 'args': {'a': 25,
'b': 12}, 'id': 'a2480228-f326-4530-876a-697fb89ba980', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens':
313, 'output_tokens': 204, 'total_tokens': 517})
```

```
In [18]:
response.tool_calls

Out [18]:
[{'name': 'multiply',
'args': {'a': 25, 'b': 12},
'id': 'a2480228-f326-4530-876a-697fb89ba980',
'type': 'tool_call'}]
```

```
In [19]:
print("LLM requested:")
print(response.tool_calls)

# Execute the requested tool
tool_call = response.tool_calls[0]

tool_name = tool_call["name"]
tool_args = tool_call["args"]

for tool in tools:
    if tool.name == tool_name:
        tool_result = tool.invoke(tool_args)
        break

print("\nTool result:")
print(tool_result)

Out [19]:
LLM requested:
[{'name': 'multiply', 'args': {'a': 25, 'b': 12}, 'id': 'a2480228-f326-4530-876a-697fb89ba980', 'type': 'tool_call'}]

Tool result:
300.0
```

5. base tool

19 de septiembre de 2026



Contenido

3. BaseTool — Subclassing for Full Control

1. Simple Definition
2. Why Does It Exist?
 - Before (function-based tools can't easily hold state)
 - After (subclass holds the connection)
3. Real-Life Analogy
4. Where It Fits in LangChain Architecture
5. Internal Working
6. What you implement / configure
 - name (attribute)
 - description (attribute)
 - args_schema (attribute)
 - _run (method) — REQUIRED
 - _arun (method) — optional async
 - custom state (extra attributes)
7. Accessing run context (callbacks / config)
- Full example
- When to subclass vs. use @tool

3. BaseTool — Subclassing for Full Control

The **most powerful** (and most manual) way to make a tool: subclass `BaseTool`. You reach for this when a decorator or `from_function` isn't flexible enough.

1. Simple Definition

Kid version: In `tool_decorator` and `structured_tool` notebook you used pre-made tool-makers (a sticker, a labeler). Subclassing `BaseTool` is like **building the tool from scratch in your own workshop** — you decide exactly how every part behaves. More work, but total control.

Professional definition: `BaseTool` is the abstract base class every tool inherits from. By subclassing it and implementing `_run` (and optionally `_arun`), you define a tool with full control over its schema, execution, state, and lifecycle.

```
from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

class MultiplyInput(BaseModel):
    a: int = Field(description="first number")
    b: int = Field(description="second number")

class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers together."
    args_schema: Type[BaseModel] = MultiplyInput

    def _run(self, a: int, b: int) -> int:
        return a * b
```

```
tool = MultiplyTool()
tool.invoke({"a": 6, "b": 7}) # 42
```

2. Why Does It Exist?

The problem: `@tool` and `StructuredTool` cover ~90% of cases, but sometimes you need:

- **State** — the tool holds a resource (a DB connection, an API client, a counter).
- **Custom logic** in construction, validation, or how args are parsed.
- **Both sync and async** with shared setup.
- To integrate a tool into a class hierarchy / reuse behavior across many tools.

Before (function-based tools can't easily hold state)

```
@tool
def query_db(sql: str) -> str:
    conn = create_connection() # ❌ new connection on EVERY call – wasteful
    return conn.execute(sql)
```

After (subclass holds the connection)

```
class DBQueryTool(BaseTool):
    name: str = "query_db"
    description: str = "Run a SQL query."
    db_conn: object # ← reused state

    def _run(self, sql: str) -> str:
        return self.db_conn.execute(sql)

tool = DBQueryTool(db_conn=create_connection()) # created ONCE, reused
```

The subclass can carry attributes (like `db_conn`) that persist across calls.

3. Real-Life Analogy

Hiring a specialist with their own equipment 🧑‍🔧. A function-tool is a temp worker who brings nothing. A `BaseTool` subclass is a specialist who **arrives with their own toolkit and workspace** (state) already set up — so every job is faster and they can do things a temp can't.

4. Where It Fits in LangChain Architecture

```
Runnable
|
▼
BaseTool ← abstract base of ALL tools
├─ StructuredTool
├─ Tool
└─ YourCustomTool ← you implement _run / _arun
```

Everything — `@tool`, `StructuredTool`, `Tool` — ultimately *is* a `BaseTool`. Subclassing puts you at the base level with maximum flexibility. Because `BaseTool` is a `Runnable`, tools get `.invoke()`, `.ainvoke()`, `.batch()`, streaming, and LCEL composition for free.

5. Internal Working

```
tool.invoke({"a": 6, "b": 7})
|
▼
BaseTool.invoke → parse & VALIDATE args against args_schema (Pydantic)
|
▼
```

```
calls YOUR _run(a=6, b=7)      (or _arun for async via .ainvoke)
|
▼
returns the result (optionally wrapped based on response_format)
```

You only write `_run`; `BaseTool` handles validation, the `Runnable` interface, callbacks, and error plumbing.

6. What you implement / configure

`name` (attribute)

Definition: The tool's identifier the model uses to call it.

Why it exists: Required — the model references the tool by name.

```
class MyTool(BaseTool):
    name: str = "search"
```

`description` (attribute)

Definition: Human-readable purpose; sent to the model.

Why it exists: The model's main signal for *when* to use the tool.

```
description: str = "Search the knowledge base. Use for policy questions."
```

`args_schema` (attribute)

Definition: A Pydantic model defining the tool's arguments.

Why it exists: Declares/validates the inputs the model must provide.

When developers use it: Whenever the tool takes structured arguments (almost always).

```
args_schema: Type[BaseModel] = SearchInput
```

`_run` (method) — REQUIRED

Definition: The actual synchronous logic executed when the tool is called.

Why it exists: This is the tool's *body* — what it does.

Real-life use case: The specialist actually doing the job.

```
def _run(self, query: str) -> str:
    return self._search_backend(query)
```

`_arun` (method) — optional async

Definition: The asynchronous implementation, used by `.ainvoke()` / async agents.

Why it exists: For I/O-bound tools (HTTP, DB) that benefit from async.

When developers use it: In async apps; otherwise LangChain can run `_run` in a threadpool.

```
async def _arun(self, query: str) -> str:
    return await self._search_backend_async(query)
```

`custom state` (extra attributes)

Definition: Any additional fields your tool needs (clients, config, counters).

Why it exists: Reuse expensive resources across calls; the whole reason to subclass.

```
class WeatherTool(BaseTool):
    name: str = "get_weather"
```

```
description: str = "Get current weather for a city."
api_key: str # ← injected once
args_schema: Type[BaseModel] = WeatherInput

def _run(self, city: str) -> str:
    return call_weather_api(city, self.api_key)

tool = WeatherTool(api_key="sk-...") # state set at construction
```

7. Accessing run context (callbacks / config)

`_run` can accept a `run_manager` for callbacks (logging, streaming progress from inside a tool):

```
from langchain_core.callbacks import CallbackManagerForToolRun

class MyTool(BaseTool):
    name: str = "long_task"
    description: str = "Run a long task."

    def _run(self, x: str, run_manager: CallbackManagerForToolRun | None = None) -> str:
        if run_manager:
            run_manager.on_text("starting...")
        return do_work(x)
```

Full example

```
from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

class SearchInput(BaseModel):
    query: str = Field(description="the search query")
    top_k: int = Field(default=3, description="number of results")

class KnowledgeBaseTool(BaseTool):
    name: str = "kb_search"
    description: str = "Search the internal knowledge base. Use for product/policy questions."
    args_schema: Type[BaseModel] = SearchInput
    index: object # a vector store / search client (state)

    def _run(self, query: str, top_k: int = 3) -> str:
        hits = self.index.search(query, k=top_k)
        return "\n".join(h.text for h in hits)

    async def _arun(self, query: str, top_k: int = 3) -> str:
        hits = await self.index.asearch(query, k=top_k)
        return "\n".join(h.text for h in hits)

tool = KnowledgeBaseTool(index=my_vectorstore)
```

When to subclass vs. use @tool

Need	Use
Simple function → tool	@tool
Third-party / dynamic function	StructuredTool.from_function
Persistent state (DB/API client)	subclass BaseTool
Custom validation / lifecycle	subclass BaseTool

Need	Use
Reuse behavior across many tools	<code>subclass</code> <code>BaseTool</code>

90% of the time `@tool` is enough. Subclass only when you genuinely need state or control — don't over-engineer.

```
In [1]:
from langchain_core.tools import BaseTool
from langchain_ollama import ChatOllama
from pydantic import BaseModel, Field
from typing import Type

# Initialize the Language model
llm = ChatOllama(model="qwen3:8b")

Out [1]:
d:\Dev_Workspace\LangChain\langchainenv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
In [2]:
# step 1: define the tool with schema
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")

class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "multiply two numbers"
    args_schema: Type[BaseModel] = MultiplyInput

    def _run(self, a: int, b:int)->int:
        return a*b

multiply_tool = MultiplyTool()

Out [2]:
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_2776\100554012.py:3: PydanticDeprecatedSince20: Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  a: int = Field(required=True, description="The first number to add")
C:\Users\Mr. Sachin Kapoor\AppData\Local\Temp\ipykernel_2776\100554012.py:4: PydanticDeprecatedSince20: Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.13/migration/
  b: int = Field(required=True, description="The second number to add")
```

```
In [3]:
print("=" * 40)
print("🔧 Tool Information")
print("=" * 40)
print(f"Name      : {multiply_tool.name}")
print(f"Description : {multiply_tool.description}")
print(f"Arguments   : {multiply_tool.args}")
print("=" * 40)

Out [3]:
=====
🔧 Tool Information
=====
Name      : multiply
Description : multiply two numbers
Arguments   : {'a': {'description': 'The first number to add', 'title': 'A', 'type': 'integer'}, 'b': {'description': 'The second number to add', 'title': 'B', 'type': 'integer'}}
=====
```

```
In [4]:
multiply_tool.args_schema.model_json_schema()

Out [4]:
```

```
{'properties': {'a': {'description': 'The first number to add',
  'required': True,
  'title': 'A',
  'type': 'integer'},
  'b': {'description': 'The second number to add',
  'required': True,
  'title': 'B',
  'type': 'integer'}},
  'required': ['a', 'b'],
  'title': 'MultiplyInput',
  'type': 'object'}
```

```
In [5]:
multiply_tool.invoke({"a": 5, "b": 3})
```

```
Out [5]:
15
```

```
In [6]:
#step2: bind the tool to the language model
llm_with_tools = llm.bind_tools([multiply_tool])
```

```
In [7]:
# step3: tool calling
msg = llm_with_tools.invoke("What's 12 times 7?")
msg
```

```
Out [7]:
AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:17:58.1035201Z',
  'done': True, 'done_reason': 'stop', 'total_duration': 10529189000, 'load_duration': 1540900, 'prompt_eval_count': 158,
  'prompt_eval_duration': 64772000, 'eval_count': 150, 'eval_duration': 10437802000, 'logprobs': None, 'model_name': 'qwen3:8b',
  'model_provider': 'ollama'}, id='lc_run--01a0b999-ec35-7b22-b2eb-f33c16e0c2b9-0', tool_calls=[{'name': 'multiply', 'args': {'b': 7,
  'a': 12}, 'id': '8daa785f-7567-4055-aab2-05443957bc19', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens':
  158, 'output_tokens': 150, 'total_tokens': 308})
```

```
In [8]:
msg.tool_calls
```

```
Out [8]:
[{'name': 'multiply',
  'args': {'b': 7, 'a': 12},
  'id': '8daa785f-7567-4055-aab2-05443957bc19',
  'type': 'tool_call'}]
```

```
In [9]:
# step 4: tool execution
result = multiply_tool.invoke(llm_with_tools.invoke("can you multiply 3 with 10").tool_calls[0]['args'])
result
```

```
Out [9]:
30
```

```
In [10]:
from typing import Type
from pydantic import BaseModel, Field
from langchain_core.tools import BaseTool
from langchain_ollama import ChatOllama
```

```
In [11]:
# STEP 1: Define input schemas
class AddInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")

class SubtractInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")
```

```

class MultiplyInput(BaseModel):
    a: float = Field(description="First number")
    b: float = Field(description="Second number")

class DivideInput(BaseModel):
    a: float = Field(description="Numerator")
    b: float = Field(description="Denominator")

```

```

In [12]:
# STEP 2: Create BaseTool classes
class AddTool(BaseTool):
    name: str = "add"
    description: str = "Add two numbers."
    args_schema: Type[BaseModel] = AddInput

    def _run(self, a: float, b: float) -> float:
        return a + b

class SubtractTool(BaseTool):
    name: str = "subtract"
    description: str = "Subtract b from a."
    args_schema: Type[BaseModel] = SubtractInput

    def _run(self, a: float, b: float) -> float:
        return a - b

class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers."
    args_schema: Type[BaseModel] = MultiplyInput

    def _run(self, a: float, b: float) -> float:
        return a * b

class DivideTool(BaseTool):
    name: str = "divide"
    description: str = "Divide a by b."
    args_schema: Type[BaseModel] = DivideInput

    def _run(self, a: float, b: float) -> float:
        if b == 0:
            return "Cannot divide by zero"

```

```

In [13]:
# STEP 3: Create tool instances
add_tool = AddTool()
subtract_tool = SubtractTool()
multiply_tool = MultiplyTool()
divide_tool = DivideTool()

```

```

In [14]:
tools = [add_tool, subtract_tool, multiply_tool, divide_tool]

llm = ChatOllama(model="qwen3:8b")

```

```

In [15]:
# STEP 4: Bind tools to LLM
llm_with_tools = llm.bind_tools(tools)

```

```

In [16]:
for tool in tools:
    print("=" * 50)
    print(f"Tool       : {tool.name}")
    print(f"Description: {tool.description}")
    print(f"Arguments  : {tool.args}")
    print(f"Schema JSON:")

```

```
print(tool.args_schema.model_json_schema())
print("=" * 50)
```

Out [16]:

```
=====
Tool      : add
Description: Add two numbers.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'AddInput', 'type': 'object'}
=====
Tool      : subtract
Description: Subtract b from a.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'SubtractInput', 'type': 'object'}
=====
Tool      : multiply
Description: Multiply two numbers.
Arguments  : {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'First number', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Second number', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'MultiplyInput', 'type': 'object'}
=====
Tool      : divide
Description: Divide a by b.
Arguments  : {'a': {'description': 'Numerator', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Denominator', 'title': 'B', 'type': 'number'}}
Schema JSON:
{'properties': {'a': {'description': 'Numerator', 'title': 'A', 'type': 'number'}, 'b': {'description': 'Denominator', 'title': 'B', 'type': 'number'}}, 'required': ['a', 'b'], 'title': 'DivideInput', 'type': 'object'}
=====
```

In [17]:

```
# STEP 5: Invoke the LLM with tools
response = llm_with_tools.invoke("What is 25 multiplied by 12?")
response
```

Out [17]:

```
AIMessage(content='', additional_kwargs={}, response_metadata={'model': 'qwen3:8b', 'created_at': '2026-09-19T12:18:16.880707Z', 'done': True, 'done_reason': 'stop', 'total_duration': 10863766000, 'load_duration': 3625100, 'prompt_eval_count': 313, 'prompt_eval_duration': 79999000, 'eval_count': 145, 'eval_duration': 10757175000, 'logprobs': None, 'model_name': 'qwen3:8b', 'model_provider': 'ollama'}, id='lc_run--01a0b99a-343f-7b83-ab40-b7c7a1da62f4-0', tool_calls=[{'name': 'multiply', 'args': {'a': 25, 'b': 12}, 'id': '2d852f61-7ad7-48b5-8921-ca2adca67db1', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 313, 'output_tokens': 145, 'total_tokens': 458})
```

In [18]:

```
response.tool_calls
```

Out [18]:

```
[{'name': 'multiply',
  'args': {'a': 25, 'b': 12},
  'id': '2d852f61-7ad7-48b5-8921-ca2adca67db1',
  'type': 'tool_call'}]
```

In [19]:

```
print("LLM requested:")
print(response.tool_calls)

# Execute the requested tool
tool_call = response.tool_calls[0]

tool_name = tool_call["name"]
tool_args = tool_call["args"]
```

```
for tool in tools:
    if tool.name == tool_name:
        tool_result = tool.invoke(tool_args)
        break

print("\nTool result:")
print(tool_result)
```

Out [19]:

LLM requested:

```
[{'name': 'multiply', 'args': {'a': 25, 'b': 12}, 'id': '2d852f61-7ad7-48b5-8921-ca2adca67db1', 'type': 'tool_call'}]
```

Tool result:

300.0